

```

/**
 *
=====
=====
 * index.ts – الدمج النهائي بين الجزء الأول – v3
layers) (مربوط فعلياً v2) والجزء الثاني
 *
=====
=====
 * artifacts/api-
server/src/lib/ai/index.ts (أو أي مسار تحبه)
 * POST /api/chat من مسار handleChat واستدع
مباشرة.
 */

import { ToolContext } from "../tool-
orchestrator-v2";
import { chatWithToolsRobust,
streamFinalResponse, getSystemMetrics }
from "../tool-orchestrator-v3-layers";

interface IncomingChatMessage { role:
"system" | "user" | "assistant" | "tool";
content: string; }

/**
 * نقطة الدخول الوحيدة التي تحتاجها في راوت
/api/chat.
 * بتجمع: rate limiting + cache + fallback +
تنفيذ الأدوات الحقيقي كله في استدعاء واحد.

```

```

*/
export async function handleChat(params: {
  userId: string;
  accountId: string;
  apiToken: string;
  preferredModel: string;
  messages: IncomingChatMessage[];
  githubToken?: string;
  getFileBuffer?: (fileId: string) =>
Promise<Buffer>;
}) {
  const ctx: ToolContext = {
    userId: params.userId,
    accountId: params.accountId,
    apiToken: params.apiToken,
    extras: { githubToken:
params.githubToken, getFileBuffer:
params.getFileBuffer },
  };

  return chatWithToolsRobust(ctx,
params.preferredModel, params.messages, {
useCache: true });
}

/** لو محتاج تبث الرد تدريجياً على الواجهة SSE نسخة */
export async function*
handleChatStream(params: Parameters<typeof
handleChat>[0]) {
  const ctx: ToolContext = {
    userId: params.userId,
    accountId: params.accountId,
    apiToken: params.apiToken,
    extras: { githubToken:

```

```

params.githubToken, getFileBuffer:
params.getFileBuffer },
  };
  yield* streamFinalResponse(ctx,
params.preferredModel, params.messages);
}

export { getSystemMetrics };

/**
 * مثال استخدام فعلي في Express:
 *
 * import { handleChat } from "../lib/ai";
 *
 * app.post("/api/chat", async (req, res)
=> {
  * try {
  *   const result = await handleChat({
  *     userId: req.user.id,
  *     accountId:
process.env.CLOUDFLARE_ACCOUNT_ID!,
  *     apiToken:
process.env.CLOUDFLARE_API_TOKEN!,
  *     preferredModel: "@cf/meta/llama-
3.3-70b-instruct-fp8-fast",
  *     messages: req.body.messages,
  *     githubToken:
req.user.githubToken, // لو عندك
تكمّل GitHub
  *     getFileBuffer: (fileId) =>
storage.readFile(fileId), // اربطها بتخزينك
الفعلي
  *   });
  *   res.json(result);

```

```
*   } catch (err: any) {  
*       res.status(err.name ===  
"RateLimitError" ? 429 : 500).json({ error:  
err.message });  
*   }  
* });  
*/
```